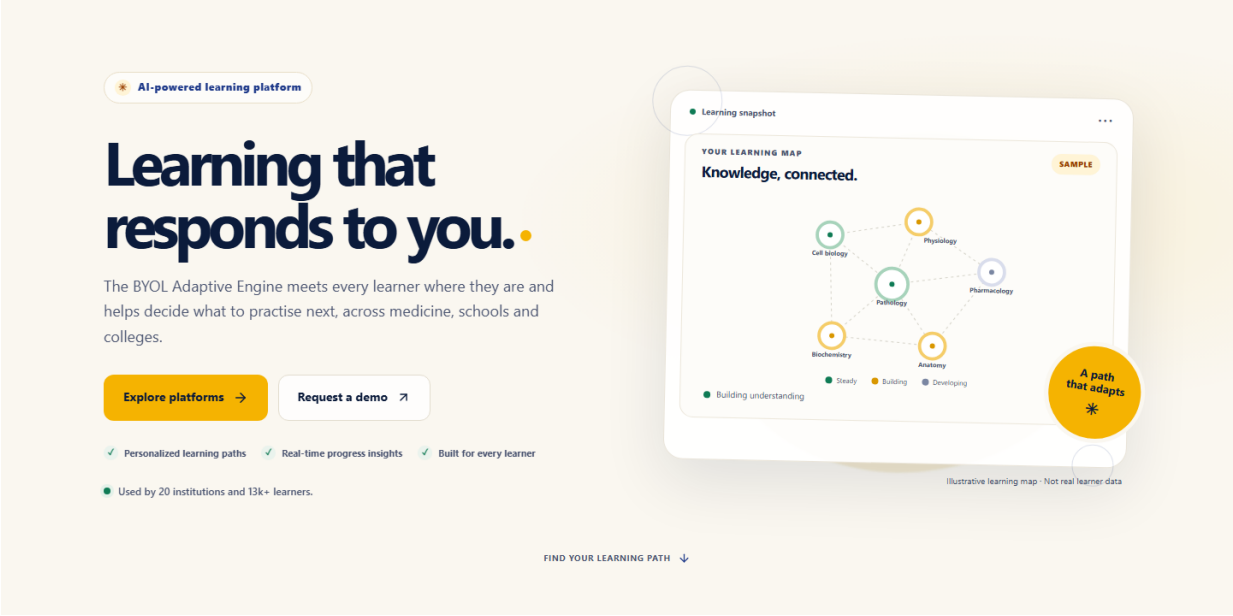
BYOL ACADEMY

Your website, explained from the ground up

A beginner-friendly tour of the design, structure, pages, code, interactive demo, tests, and steps needed before launch.



Screenshot 1. Homepage hero: the first message, learning-map illustration and visitor paths.

> How to use this guide
> Read it in order if you are new to websites. You do not need programming knowledge first. Code examples are small maps to the real files, not code you need to paste.

Based on the supplied BYOL Academy build guide and the website source - 25 October 2026

A map of this guide

This PDF explains what was built, how to explore it, and what each part of the code is responsible for. It includes screenshots from the local production build.

Chapter	What you will learn
1. Start here	The request and the steps taken to build the site.
2. Structure	Routes, components, content, libraries and configuration.
3. Shared frame	Layout, navigation, footer and metadata.
4. Homepage	Sections, audiences, product cards, trust and calls to action.
5. Interactive demo	States, questions, mastery calculation and accessibility.
6. Other pages	About/research, contact, blog, legal pages and styleguide.
7. Forms and safety	Validation, rate limiting, email and why contact is disabled.
8. Design and SEO	Responsive CSS, search metadata and accessibility.
9. Files and tests	Where code lives and what was verified.
10. Run and launch	Commands, environment settings and launch dependencies.
11. Glossary	Common web terms explained plainly.

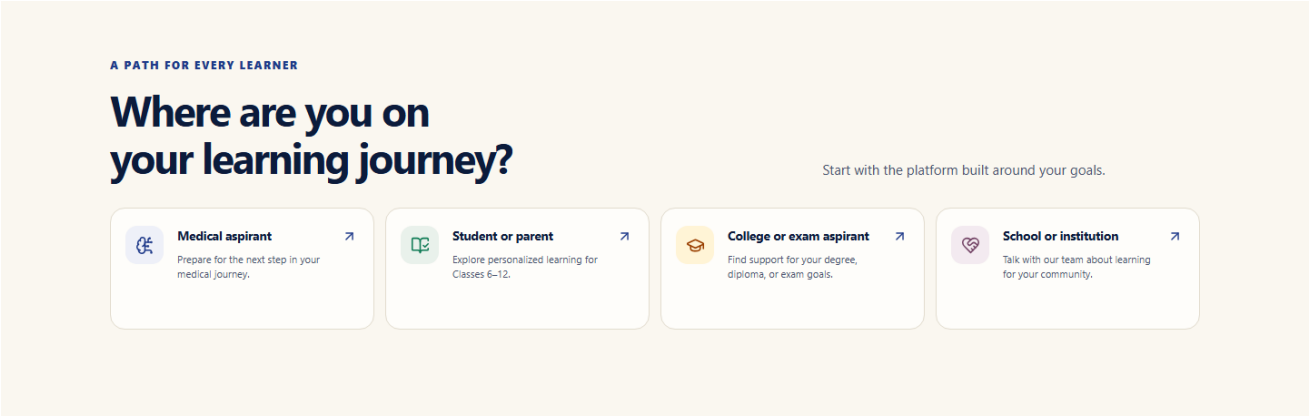
Important boundary
The learning demo is an illustrative sample only. It is not BYOL's deployed production engine and does not use real learner information.

1. What was built and how I approached it

You asked for a complete, production-style BYOL Academy website based on the supplied guide. I first read the build guide and adaptive-learning research brief, then created the site, added tests and checked the result in a browser.

The work, step by step

Step	What happened	Why it matters
1. Read the brief	Extracted page requirements, approved content, proposed stack and safeguards.	Avoids invented claims, logos and legal copy.
2. Set up the project	Created a pinned Next.js, React and TypeScript app with scripts and configuration.	Makes install and build repeatable.
3. Add shared content	Created typed platform data, site copy, demo questions and approval flags.	Keeps pages consistent and editable.
4. Build common frame	Added announcement, responsive navigation, footer, metadata and design tokens.	All routes have consistent behavior.
5. Build the pages	Added homepage and product pathways, research, demo, contact and supporting routes.	Visitors get relevant next steps.
6. Add safeguards	Validated server input, origin, size, bot field and optional rate limits. Kept form off.	A browser form alone is not safe lead delivery.
7. Add SEO/accessibility	Added metadata, sitemap, robots, keyboard support and reduced-motion styles.	Helps people and crawlers use the site.
8. Verify	Ran lint, typecheck, tests, browser/axe, build, audit and route smoke checks.	Checks code and rendered behavior.



Screenshot 2. Audience choices help people find the appropriate product path.

What “production-level” means here

The project has a working production build, reusable code, tests, route metadata and a protected lead-form backend. That does not mean the business can launch without approvals: legal documents, brand assets, permissions, production accounts and hosting configuration still belong to BYOL Academy.

> The site is real; some connections are intentionally gated
> Privacy and Terms are placeholders. The contact form remains disabled until consent is approved and email/rate-limit services are configured. Analytics stays disabled pending privacy review.

2. How the website is put together

Think of the site as layers. A visitor opens a URL (a route); Next.js selects the page file; React renders reusable components; components use content files; CSS determines how the result looks at different screen sizes.

Layer	Beginner comparison	Example
Routes	Rooms with different addresses	/about, /contact, /privacy
Components	Reusable furniture	Header, footer, interactive demo
Content	Labels/information on the furniture	src/content/site.ts
Styles	Colors, spacing, arrangement	src/styles/globals.css
Helpers	Specialist repeated tasks	Zod validation, analytics, mastery calculation
Tests	Computer-rerunnable checklist	tests/unit and tests/e2e
Deployment config	Instructions to build/publish	package.json, next.config.ts, CI workflow

Project folder map

```
byol-academy-web/
  src/
    app/                    Website routes and page-level files
      page.tsx              Homepage (/)
      layout.tsx            Shared outer frame
      about/page.tsx        About and research (/about)
      contact/page.tsx      Contact (/contact)
      privacy/page.tsx      Privacy placeholder
      terms/page.tsx        Terms placeholder
      api/lead/route.ts     Server endpoint for enquiries
      robots.ts             Search-crawler instructions
      sitemap.ts            Search-engine page list
    components/             Reusable interface building blocks
    content/                Copy, product details, sample questions
    lib/                    Calculations, analytics, SEO, validation
    styles/globals.css      Design system and responsive CSS
  tests/unit/               Small isolated logic tests
  tests/e2e/                Real-browser flows and accessibility checks
  package.json              Commands and dependencies
  next.config.ts            Next.js settings and security headers
  README.md                 Setup instructions
  LAUNCH_CHECKLIST.md       Requirements and launch blockers
```

How one visitor page is assembled

```
Browser visits /
       |
       v
src/app/page.tsx (homepage sections)
       | uses
       +--> Header.tsx + Footer.tsx (shared navigation)
       +--> AdaptiveDemo.tsx (interactive sample)
       +--> src/content/site.ts (approved words and links)
       +--> src/styles/globals.css (layout and colors)
       |
       v
HTML + CSS + interactive JavaScript
```

The homepage composes some simple sections directly. More interactive or reusable features live in their own component files.

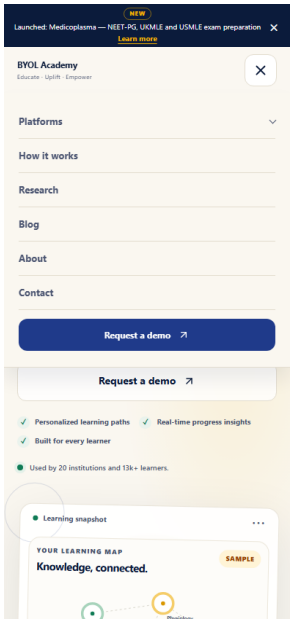
3. The shared frame: layout, header and footer

src/app/layout.tsx wraps every route. It is like a common picture frame: document language, metadata, skip link, header, main content, footer and optional analytics provider.

```
<html lang="en-IN">
  <body>
    <a className="skip-link" href="#main">Skip to content</a>
    <Header />
    <main id="main">{children}</main>
    <Footer />
    {analyticsEnabled && <Analytics />}
  </body>
</html>
```

The `children` value is the route page placed inside the frame. The language setting helps browsers and assistive technology pronounce content as Indian English.

Navigation on phones



Screenshot 3. At a narrow width, navigation becomes a vertical menu and keeps the demo action prominent.

Header.tsx is interactive, so it uses the React client component marker “use client”. A small menuOpen state determines whether the mobile links are visible. aria-expanded tells assistive technology whether the menu is open. Escape closes it and returns keyboard focus to the button. The platform dropdown uses native HTML details/summary, which supports keyboard interaction.

Other shared pieces

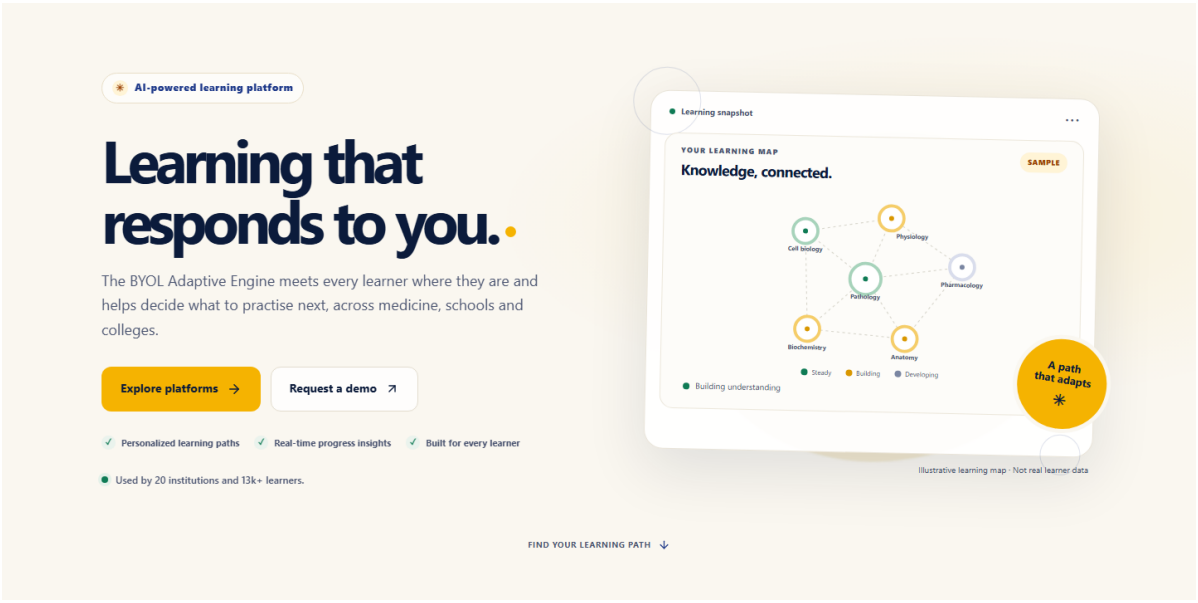
File	What it does
AnnouncementBar.tsx	Dismissible launch message; remembers dismissal in sessionStorage for this tab session.
Footer.tsx	Repeats platform, company and legal destinations across routes.
JsonLd.tsx	Adds structured Organization/Website information for crawlers; escapes script-sensitive text.
src/app/layout.tsx	Default title, description, canonical and social preview metadata.
src/app/opengraph-image.tsx	Generates the social-sharing preview image.

Metadata is the page label for search and sharing

Each page can set its own title and description. The shared layout supplies defaults, the canonical site URL and social-card information. Metadata helps describe a page; it does not guarantee search ranking.

4. Homepage: the sections visitors see

src/app/page.tsx composes the homepage from top to bottom. It reads much of its wording and destination data from src/content/site.ts. Repeated cards are made by looping over arrays, rather than copying the same markup by hand.



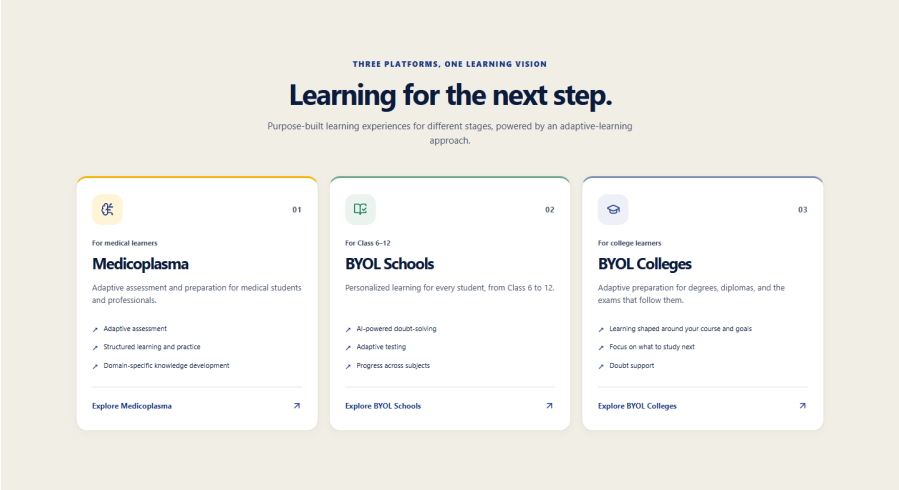
Screenshot 4. Hero section: headline, mastery-map illustration and routes for different visitors.

Homepage section	Purpose	Implementation
Announcement + header	Promote launch and expose navigation.	AnnouncementBar.tsx, Header.tsx
Hero	Explain the product and offer next steps.	hero markup in app/page.tsx
Audience chooser	Route medical, school, college and institution visitors.	audiences in site.ts
Product cards	Summarize the three learning products.	platform data + card markup
Learning approach	Explain benefits of responsive learning.	benefits array
Engine loop	Show a conceptual five-step process.	engineSteps array
See it adapt	Let a visitor try an offline sample.	AdaptiveDemo.tsx
Proof	Show only approved numbers.	stats array; no fake testimonials/logos
Responsible AI	Clarify intended use and principles.	trust copy and principles
Institution CTA	Invite a conversation about a demonstration.	Contact route with intent parameter
Closing CTA/footer	Offer next steps and site links again.	homepage + Footer.tsx

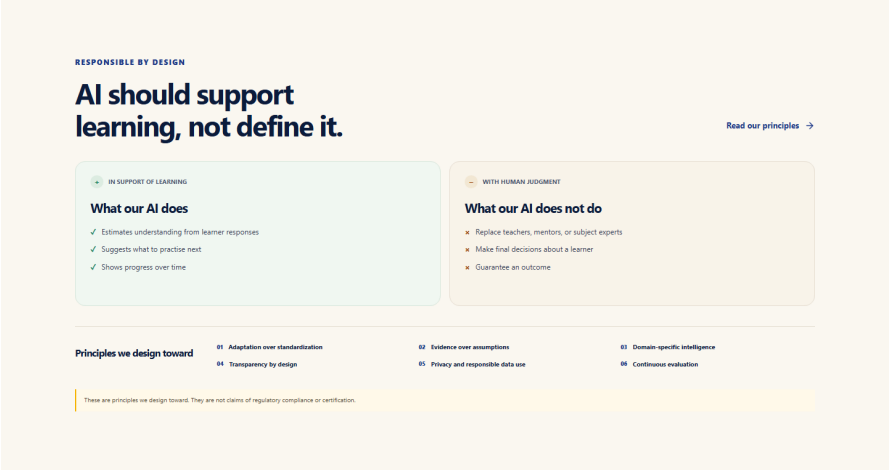
How a repeated card is created

```
platforms.map((platform) => (
  <article key={platform.id}>
    <h3>{platform.name}</h3>
    <p>{platform.summary}</p>
    <a href={platform.href}>{platform.ctaLabel}</a>
  </article>
))
```

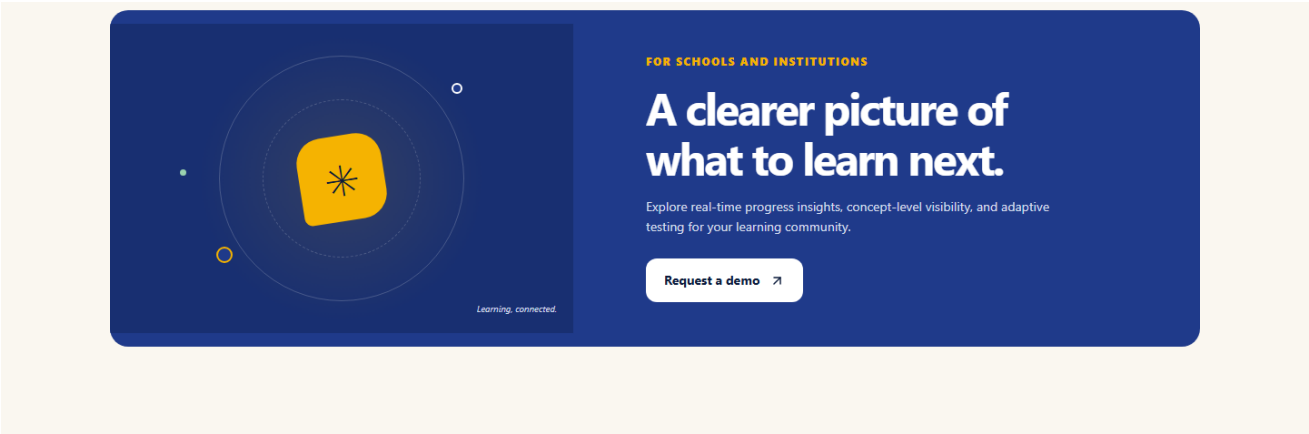
map() loops over each platform object and creates a card. The `key` is a stable identifier React uses when the list changes. The content is not duplicated manually in every card.



Screenshot 5. The three cards share a structure while product names, audiences and links vary.



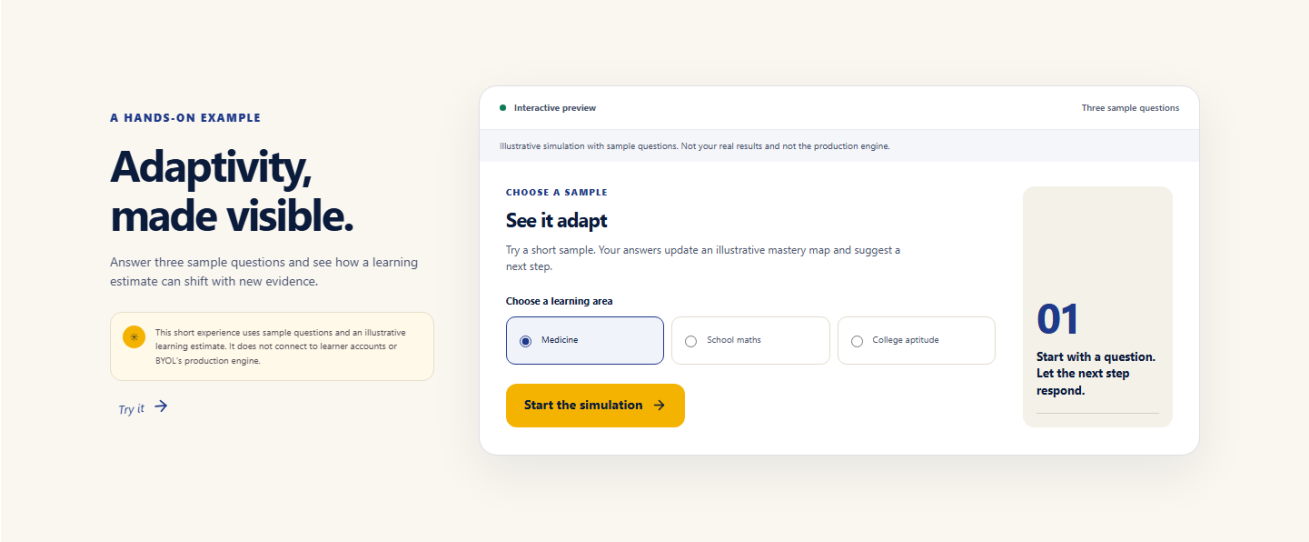
Screenshot 6. The responsible-AI section gives visitors important context about the system.



Screenshot 7. The institution call to action opens contact with institution intent.

5. Interactive demo: what changes when you answer

AdaptiveDemo.tsx is a client-side React component. It remembers the current subject, question, selection and sample estimates while a visitor clicks through.

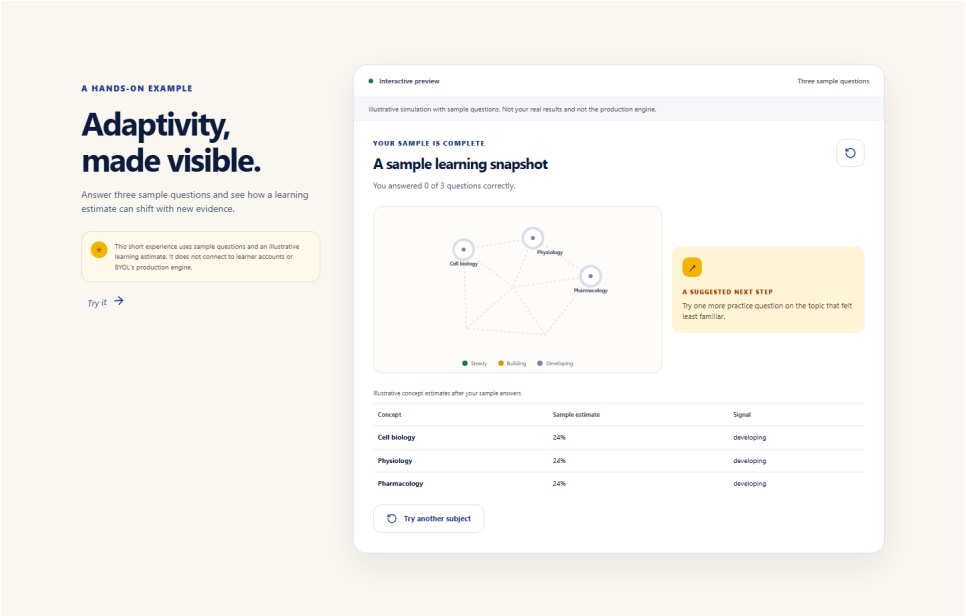


Screenshot 8. Select a sample subject before starting the learning loop.

Four states, one clear flow

State	What is visible	Next action
idle	Choose Medicine, School maths or College aptitude.	Start → asking
asking	Question, four answers and progress.	Check answer → feedback
feedback	Correct/incorrect response and explanation.	Next → asking or results
results	Illustrative mastery map, data table and next step.	Try another subject → idle

React re-renders when state changes. The component also moves keyboard focus to the next question or results heading, so a keyboard user can follow the transition.



Screenshot 9. Results include the map, an accessible table and an always-visible illustrative disclaimer.

The sample calculation in everyday language

src/lib/adaptive/mastery.ts implements an illustrative Bayesian Knowledge Tracing-style update. It starts with an estimate, allows for a guess or accidental slip, updates based on the response, then adds a small learning opportunity. Sample parameters are guess 0.20, slip 0.10 and learn 0.15.

```
Prior estimate = 50%; answer is correct
known and correct   = 0.50 * (1 - 0.10) = 0.45
unknown but guessed = 0.50 * 0.20        = 0.10
evidence total      = 0.45 + 0.10        = 0.55
posterior estimate  = 0.45 / 0.55        = 0.818
add learning chance = 0.818 + (1-.818)*.15
new sample estimate                       about 0.845
```

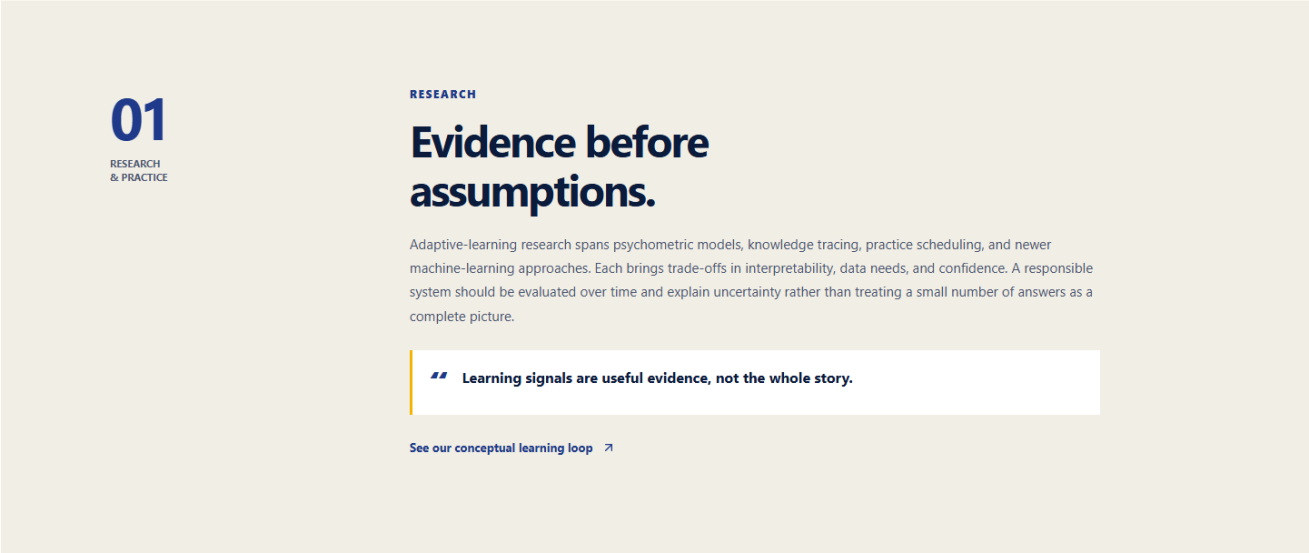
A wrong answer is not treated as absolute proof of no knowledge. Guess/slip terms allow for uncertainty. Labels such as developing, building and steady are demo categories, not grades or formal assessments.

> Honest-AI rule
> This is an offline illustration with sample questions. It is not connected to accounts, and is not the live BYOL engine or a validated learner assessment.

6. Other pages and their purpose

Route	Source file	Purpose
/	src/app/page.tsx	Homepage and product pathways.
/about	src/app/about/page.tsx	Vision, conceptual engine, research and responsible-AI principles.
/contact	src/app/contact/page.tsx	Contact context, product links and disabled/enabled lead form.
/privacy	src/app/privacy/page.tsx	Placeholder until legal team supplies the approved privacy notice.
/terms	src/app/terms/page.tsx	Placeholder until approved terms are supplied.
/blog	src/app/blog/page.tsx	Empty state; does not invent articles.
/styleguide	src/app/styleguide/page.tsx	Noindex page to inspect colors, buttons and typography.
Unknown URL	src/app/not-found.tsx	Helpful branded 404 page.
Runtime error	src/app/error.tsx	Retry experience without exposing form content.
/api/lead	src/app/api/lead/route.ts	Server-side lead safety and email delivery.

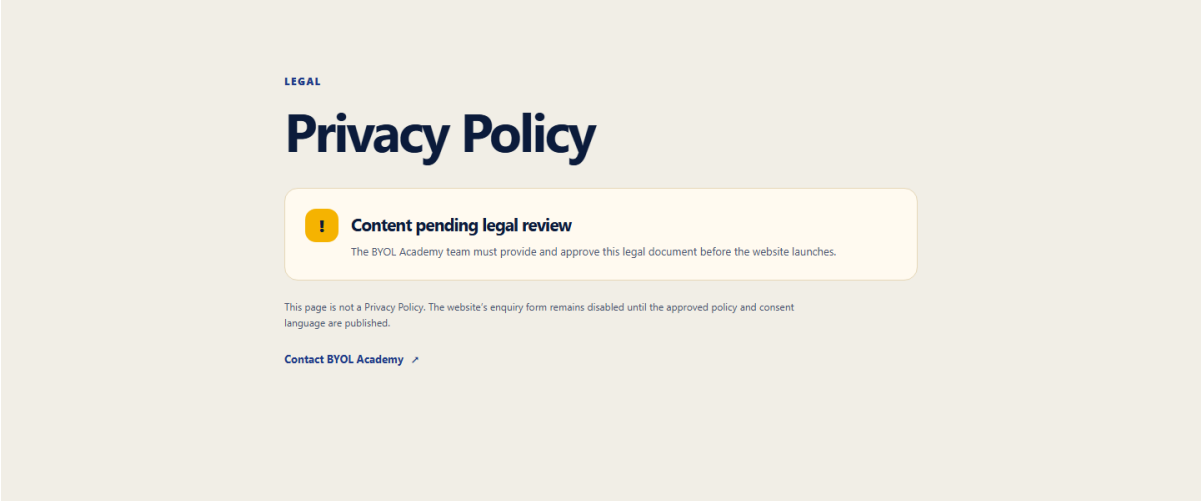
About and research



Screenshot 10. The research section describes the concept without claiming an unverified live model.

The supplied research brief informed the high-level explanations of knowledge tracing and psychometric approaches. The page makes a clear distinction between an intended design and a deployed system.

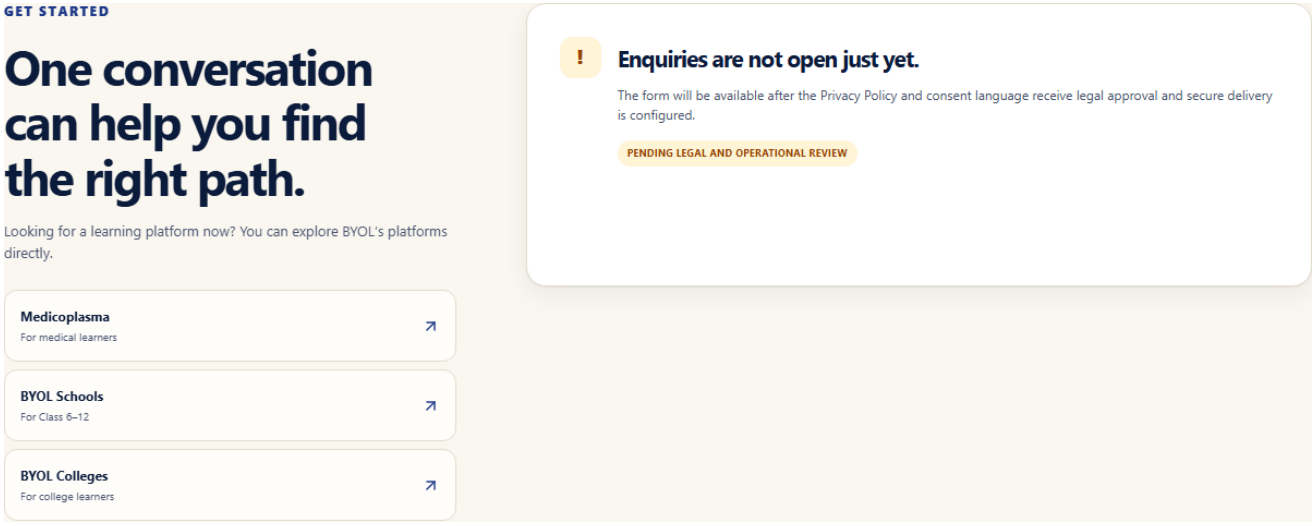
Legal pages and blog



Screenshot 11. The privacy route clearly says approved legal text is still needed.

No articles or approved legal wording were supplied. Empty/placeholder content is safer than invented posts or a made-up legal policy.

Contact context



Screenshot 12. Institution context is preselected, and the current form is visibly disabled pending approvals.

A URL such as /contact?intent=institution carries context from the homepage. The Contact page reads the parameter server-side and can preselect the right intent if the form is later enabled.

7. The contact form and its safeguards

LeadForm.tsx collects input in the browser. src/app/api/lead/route.ts runs on the server and decides whether an enquiry can be accepted and emailed.

The browser form

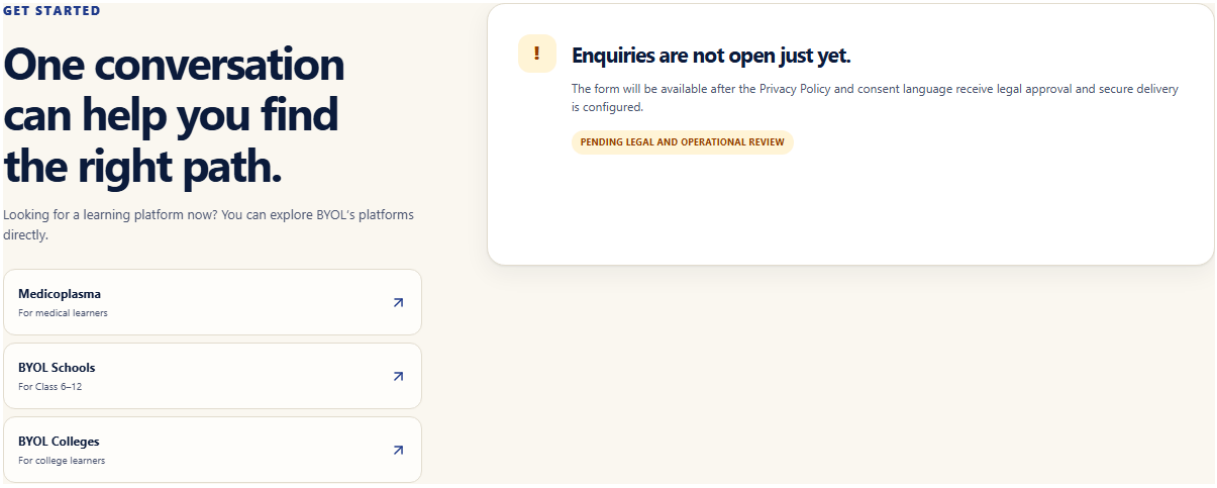
It has labels, required markers, an intent picker, an organisation field for institutions, an optional message, consent checkbox, hidden honeypot and status messages. The browser sends JSON to /api/lead only when submitted.

Server checks, in order

Check	Reason
JSON content type	Keep request format predictable.
Origin matches the site	Reject simple cross-site submissions.
Body capped at 10,000 bytes	Avoid unbounded request memory.
Zod schema validates data	Browser fields are not trusted just because a form generated them.
Honeypot inspected	Detect common automated submissions.
Flags and credentials required	Do not pretend an unconfigured email was delivered.
Upstash rate limit	Reduce repeated abuse without storing raw IP in the key.
Resend plain-text email	Keep provider keys server-side; avoid HTML injection.

```
if (LEAD_FORM_ENABLED !== "true" ||
    LEAD_CONSENT_APPROVED !== "true" ||
    !resendApiKey || !leadToEmail || !leadFromEmail ||
    !redisUrl || !redisToken) {
  return jsonError("The enquiry form is not configured yet.", 503);
}
```

This is called fail-closed: missing one setting means no submission. The browser flag controls visibility; separate server flags and credentials control real delivery. A visitor cannot cause server-side email merely by changing browser code.



Screenshot 13. Current visitors receive a clear disabled state; no enquiry can be collected yet.

> Never paste secrets into code or chat
> Set real production credentials in the hosting provider's private environment settings. Do not commit a real .env file.

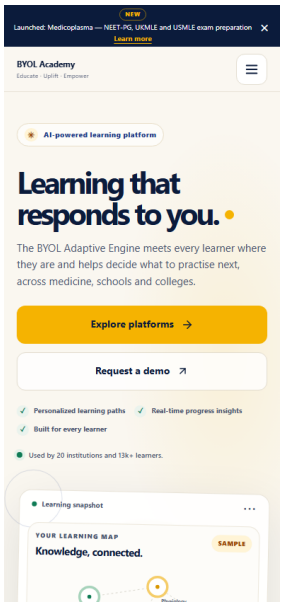
8. Design, responsive behavior, SEO and accessibility

CSS design system

src/styles/globals.css contains CSS variables, typography, page sections, cards, buttons, breakpoints, focus rings and reduced-motion handling. Variables let the team tune the palette centrally.

```
:root {
  --color-ink: #0b1b3b;
  --color-canvas: #faf7f0;
  --color-primary: #1f3a8a;
  --color-accent: #f5b301;
  --color-muted: #4a5470;
  --radius-card: 16px;
}
```

The stylesheet uses responsive breakpoints around 1050, 820 and 600 pixels. Grids become narrower, navigation becomes a menu, and content stacks instead of squeezing a desktop layout.



Screenshot 14. The homepage reflows for a narrow phone viewport.

Accessibility features

- Skip link bypasses repeated navigation.

- Menu button has an accessible name and aria-expanded state; Escape closes it and returns focus.

- Quiz answers use fieldsets and radio labels; the mastery graphic has an equivalent data table.

- Demo state changes move focus and announce feedback.

- Reduced-motion CSS turns off nonessential transitions.

- Visible keyboard focus helps people see where they are.

- Axe found zero serious/critical issues on seven routes and the completed demo state at phone/desktop sizes. This is not a full WCAG certification.

Search and social sharing

File	Search/sharing role
layout.tsx	Default titles, descriptions, canonical URL and social metadata.
opengraph-image.tsx	Branded social-sharing image.
robots.ts	Allows production crawlers, blocks API and preview deploys.
sitemap.ts	Lists public pages.
JsonLd.tsx	Structured Organization/Website description.

Analytics and HTTP headers

Analytics is gated by NEXT_PUBLIC_ANALYTICS_ENABLED and remains off unless deliberately enabled. Tracked actions include demo progress, audience selection, CTA clicks and scroll depth; it does not send names, emails or enquiry text.

next.config.ts configures HSTS, nosniff, referrer policy, disabled camera/microphone/location permissions and frame denial. Confirm HTTPS and deployment needs before relying on production headers.

> Automated checks are not a human audit
> Screen-reader use, keyboard-only interaction, zoom, phone contrast and deployed performance still need manual review.

9. File-by-file code tour

In VS Code, press Ctrl+P and type part of a filename to open it. Start with content files for copy changes and component files for behavior.

Source file/group	Responsibility
src/app/page.tsx	Homepage section order and calls to action.
src/app/layout.tsx	Shared HTML frame, metadata, header/footer and analytics.
src/app/about/page.tsx	About, conceptual engine, research and principles.
src/app/contact/page.tsx	Contact content and intent prefill.
privacy/page.tsx; terms/page.tsx	Approved-legal-content placeholders.
blog/page.tsx; styleguide/page.tsx	Empty blog and noindex design samples.
not-found.tsx; error.tsx	404 and runtime recovery.
api/lead/route.ts	Server validation, request guards, rate limit, email.
robots.ts; sitemap.ts; opengraph-image.tsx	Crawler rules, page list and sharing image.
Header.tsx; Footer.tsx; AnnouncementBar.tsx	Reusable navigation, links and launch message.
AdaptiveDemo.tsx; MasteryMap.tsx	Interactive sample, map and equivalent data table.
LeadForm.tsx	Form interface and backend request.
AnalyticsEvents.tsx; ApprovalNotice.tsx; JsonLd.tsx	Analytics, development approval badge, structured metadata.
src/content/site.ts	Types, copy, products, audiences, approved stats and principles.
src/content/demo-data.ts	Three sample subjects, each with three questions.
src/lib/adaptive/mastery.ts	Illustrative probability update and status labels.
src/lib/validation/lead.ts	Zod lead schema shared with the API.
src/lib/analytics.ts; seo.ts	Privacy-controlled tracking and SEO URL/data helpers.
src/styles/globals.css	Tokens, component styles, responsive and reduced-motion CSS.

Where do I change…?

Change	Start here
Homepage/product wording	src/content/site.ts; inspect app/page.tsx for layout.
Product URL	platforms data in src/content/site.ts.
Sample quiz	src/content/demo-data.ts.
Demo calculation	src/lib/adaptive/mastery.ts and mastery tests.
Mobile menu appearance	Header.tsx and globals.css.
Contact fields	LeadForm.tsx, validation schema and route tests.
Colors and spacing	:root tokens and globals.css.

Configuration files

package.json defines scripts and pinned dependencies; package-lock.json records the exact dependency tree. tsconfig.json sets TypeScript rules and the @/* import shortcut. eslint.config.mjs defines lint checks. vitest.config.ts and playwright.config.ts configure unit/browser tests. .env.example lists setting names without secrets. .gitignore excludes generated/private files. README.md explains setup; LAUNCH_CHECKLIST.md tracks requirements; .github/workflows/ci.yml automates checks.

10. How the site was verified

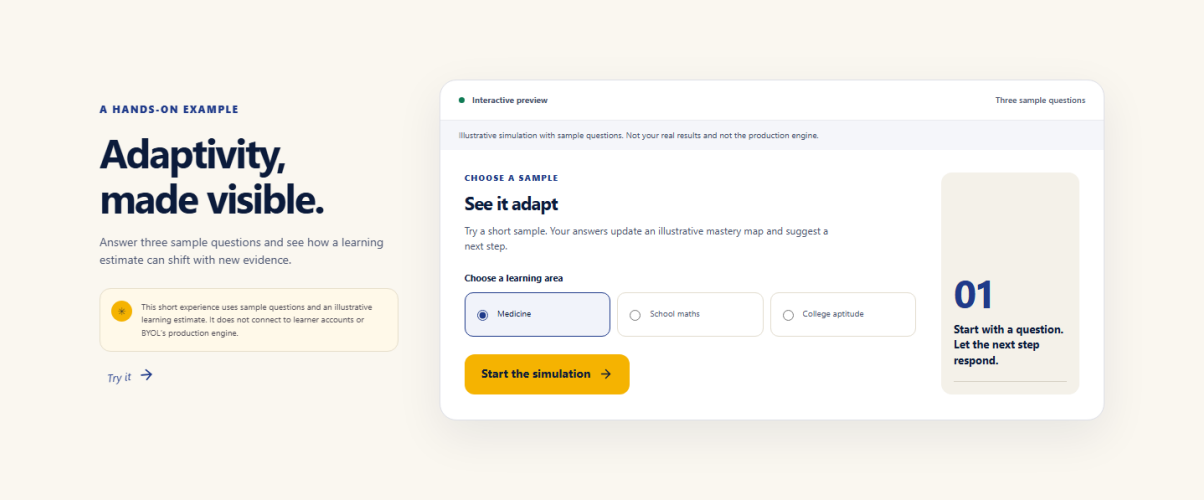
Unit tests ask a function a known question. Browser tests open real routes and click controls. A production build catches bundling/rendering issues. npm audit checks known package advisories.

Check	Result	Meaning
npm run lint	Passed	No configured ESLint violations.
npm run typecheck	Passed	Strict TypeScript check completed.
npm test	18 passed	Four files for mastery, content, form validation and API safeguards.
npm run test:e2e	21 passed; 1 skipped	Mobile/desktop Playwright and axe flows; desktop mobile-menu test intentionally skipped.
npm run build	Passed	Next.js produced production routes/assets.
npm audit --audit-level=high	0 vulnerabilities	No known findings at audit time.
HTTP smoke checks	Sample routes returned 200	Homepage, supporting pages, robots, sitemap and social image responded.

Example browser test

```
await page.goto("/#see-it-adapt");
await page.getByRole("button", { name: /start the simulation/i }).click();
await page.getByRole("radio").first().check();
await page.getByRole("button", { name: /check answer/i }).click();
await expect(page.getByRole("status")).toContainText(/right|not quite/i);
```

Playwright finds controls by accessible names and roles instead of screen coordinates. The real website is checked in both a phone-sized and desktop-sized browser.



Screenshot 15. The sample question flow used by browser tests.

Axe found no serious/critical accessibility issues in the checked pages/states. Automated tests cannot prove every part of accessibility, security or business correctness.

11. Run the website and prepare a launch

Run locally on Windows

```
# In PowerShell, open the project folder first
npm install
npm run dev
# Visit http://localhost:3000
```

npm install reads the lockfile and installs packages. npm run dev starts the local editing server; stop it with Ctrl+C. For the production version, run npm run build, then npm run start.

Environment variables

Variable	Visibility	Purpose / initial state
NEXT_PUBLIC_SITE_URL	Browser + server	Canonical site address; confirm with team.
NEXT_PUBLIC_LEAD_FORM_ENABLED	Browser + server	Show form only after approval; false.
LEAD_FORM_ENABLED	Server only	Separate API submission gate; false.
LEAD_CONSENT_APPROVED	Server only	Legal consent approval; false.
RESEND_API_KEY	Server only	Private email provider key.
LEAD_TO_EMAIL / LEAD_FROM_EMAIL	Server only	Approved recipient and sender.
UPSTASH_REDIS_REST_URL / TOKEN	Server only	Private rate-limit service.
NEXT_PUBLIC_ANALYTICS_ENABLED	Browser + server	Analytics; false until privacy review.

The team must still supply

- Approved Privacy Policy and Terms of Use.
- Legally approved contact-consent wording and collection permission.
- Verified Resend sender/domain, team inbox and server credentials.
- Upstash account/credentials for rate limiting.
- Final domain, hosting account, monitoring owner and rollback plan.
- Official logo/font/color assets if replacing the provisional text wordmark.
- Permission for any customer logos/testimonials and official social URLs.

LAUNCH_CHECKLIST.md records requirements R1-R15. Legal pages are blocked pending approved text. Accessibility/performance still require human-device review and Lighthouse checks against the deployed site.

> Publishing is a team decision
> The site is built and locally verified, but a real-domain launch needs hosting access, approved legal copy, and service credentials.

12. Beginner glossary and quick answers

Word	Plain-English meaning
App Router	Next.js method for making folders/page.tsx into website addresses.
React component	Reusable interface piece, such as Header or the demo.
TypeScript	JavaScript with extra type checks to catch mistakes early.
Server Component	Default page code rendered on the server; useful for content.
Client Component	Code marked “use client” when it needs clicks, state or browser APIs.
State	A value React remembers, such as which quiz question is active.
API route	A server URL that handles a request, such as POST /api/lead.
Environment variable	A setting provided by shell/hosting; secrets must remain server-side.
Zod schema	A readable definition and runtime checker for submitted data.
Rate limit	A cap on how often a source can call a route in a time window.
Honeypot	Hidden form field used to catch simple bots.
SEO	Search-engine optimization; descriptive structure, not ranking guarantee.
Open Graph	Metadata used when sharing a page on social platforms.
JSON-LD	Structured JSON description intended for search engines.
CI	Automated checks run when code changes.
WCAG / axe	Accessibility guidelines / an automated checker for some issues.
Lighthouse	Browser audit for performance, accessibility, SEO and best practices.
BKT	A family of models updating estimated skill from answers; used here illustratively.
LCP / INP / CLS	Web metrics for main content load, interaction response and layout shifts.

If you remember only six things

- Most copy and product links are in src/content/site.ts.
- The homepage is assembled in src/app/page.tsx.
- The shared wrapper is src/app/layout.tsx.
- Visual rules are in src/styles/globals.css.
- The sample engine is illustrative, not BYOL's live engine.
- Do not enable the contact form until approvals and credentials are ready.

A safe first edit
Change one text value in src/content/site.ts, then run npm run lint, npm run typecheck, npm test and npm run build. Inspect the local site before publishing.

End of beginner guide - Project: E:/WORK/Byol Academy - Verification recorded 25 October 2026